

§ 15. 输入输出流 - Windows与Linux的文件格式差别



要求:

- 1、安装UltraEdit软件（**附件已给，版本虽旧，但够用**），学会使用16进制方式查看文件，并掌握ASCII及16进制查看间的切换
 - ★ 已安装VSCode的也可通过相关插件进行16进制方式的查看（**VSCode某种情况下会自动做格式转换或字符集转换，要注意!!!**）
 - ★ 也可以使用其它编辑软件，但**不建议**NotePad++
- 2、完成本文档中所有的测试程序并填写运行结果，从而掌握Windows与Linux两个系统下的文本文件的差异
- 3、题目明确指定编译器外，Windows下用VS2022编译，Linux下用C++编译
 - ★ 如果要换成其他编译器，可能需要自行修改头文件适配
 - ★ 部分代码编译时有**warning**，不影响概念理解，**可以忽略**
- 4、直接在本文件上作答，**写出答案/截图（不允许手写、手写拍照截图）**即可；填写答案时，为适应所填内容或贴图，**允许调整**页面的字体大小、颜色、文本框的位置等
 - ★ 贴图要有效部分即可，不需要全部内容
 - ★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可
 - ★ **不允许**手写在纸上，再拍照贴图
 - ★ **允许**在各种软件工具上完成（不含手写），再截图贴图
 - ★ 如果两个编译器运行结果一致，贴其中一张图即可，如果不一致，则两个图都要贴
- 5、转换为pdf后提交
- 6、**11月7日前**网上提交本次作业（在“文档作业”中提交）

特别说明:

- ★ 因为篇幅问题，打开文件后均省略了是否打开成功的判断，这在实际应用中是**不允许**的

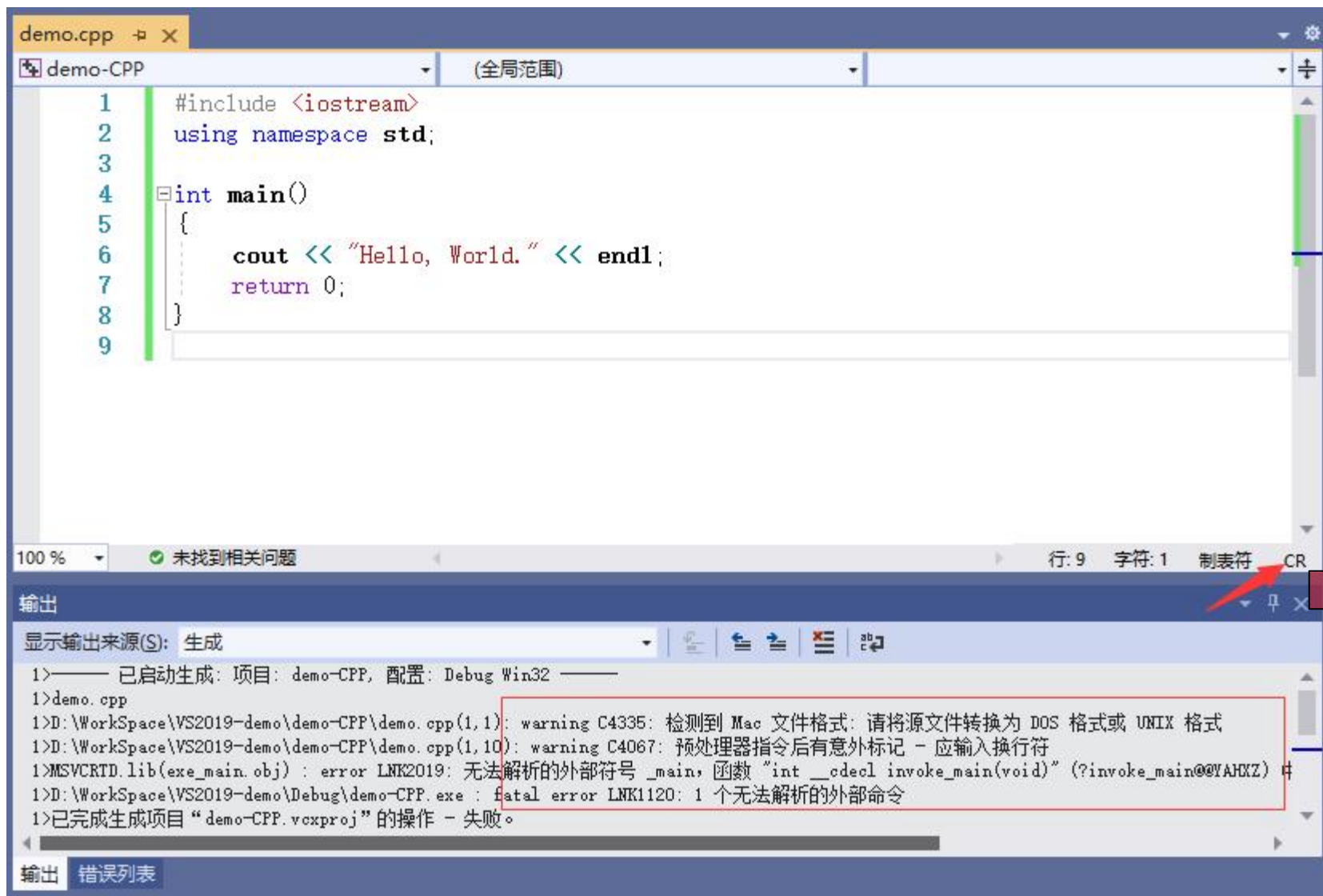


§ 15. 输入输出流 - Windows与Linux的文件格式差别

注意:

附1: 用WPS等其他第三方软件打开PPT, 将代码复制到VS2022中后, 如果出现类似下面的**编译报错**, 则观察源程序编辑窗

的右下角是否为CR, 如果是, 单击CR, 在弹出中选择CRLF, 再次CTRL+F5运行即可

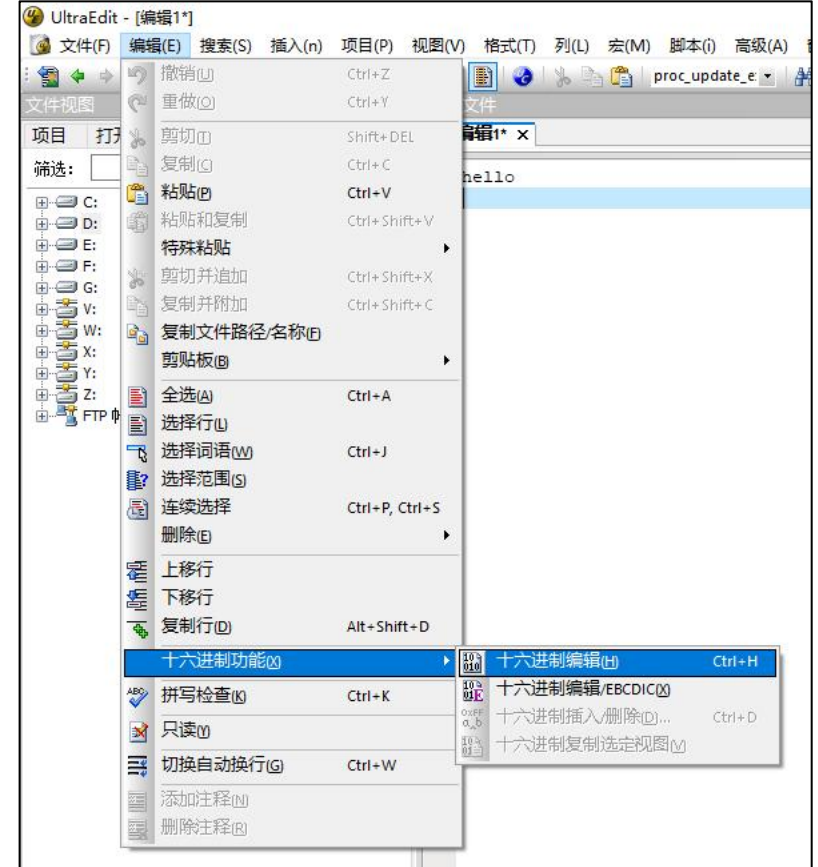
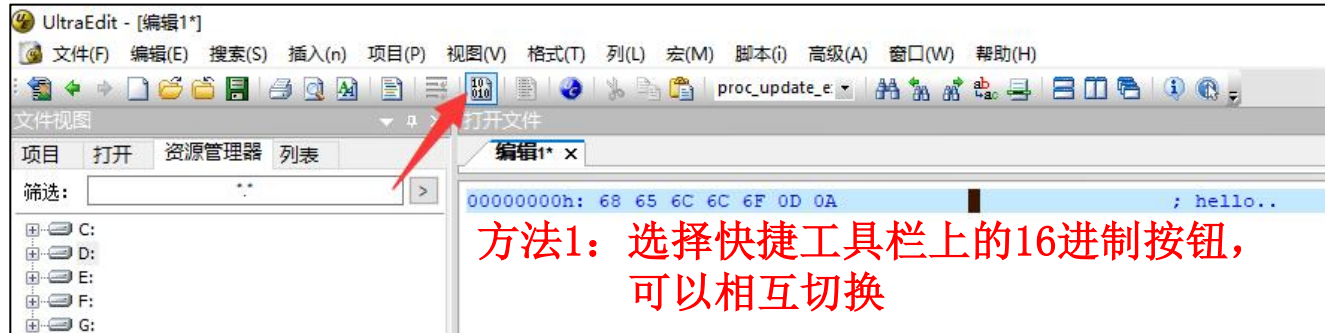
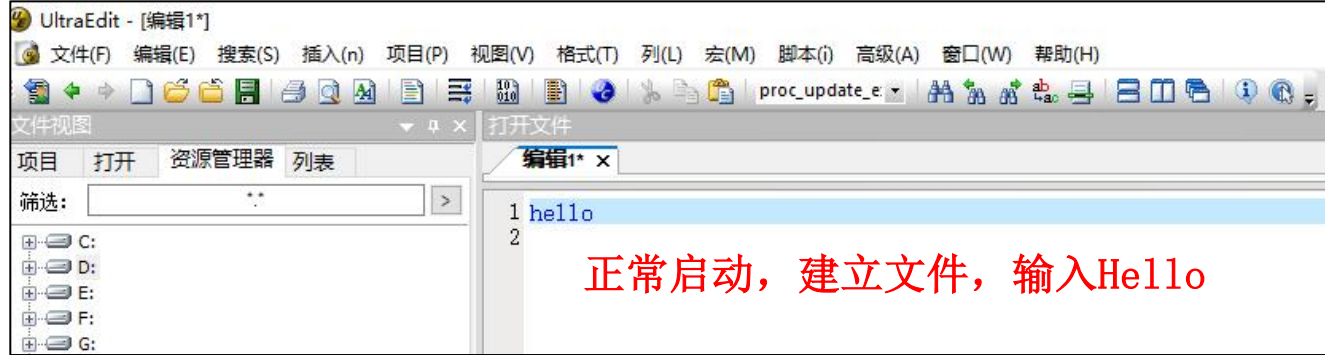




§ 15. 输入输出流 - Windows与Linux的文件格式差别

注意:

附2: 附件给出的UltraEdit查看文件的16进制形式的方法 (三种)



方法3: Ctrl + H 快捷键可以相互切换

§ 15. 输入输出流 - Windows与Linux的差别

本页需填写答案



例1: 十进制方式写, 在Windows/Linux下的差别

```
#include <iostream>
#include <fstream>
using namespace std;

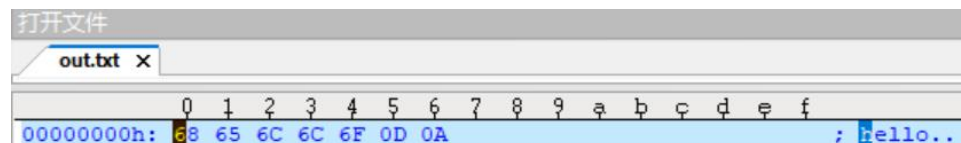
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);

    out << "hello" << endl;

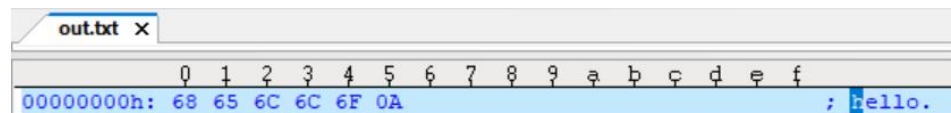
    out.close();

    return 0;
}
```

Windows下运行, out.txt是__7__字节, 用UltraEdit的16进制方式打开的贴图



Linux下运行, out.txt是__6__字节, 用UltraEdit的16进制方式打开的贴图



§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例2: 二进制方式写, 在Windows/Linux下的差别

```
#include <iostream>
#include <fstream>
using namespace std;

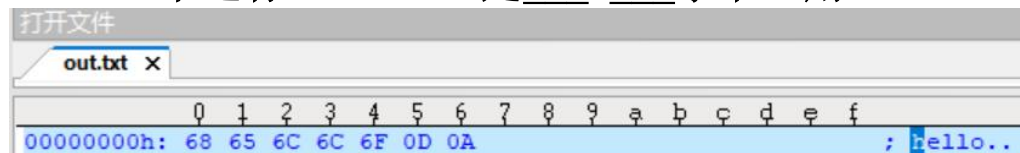
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);

    out << "hello" << endl;

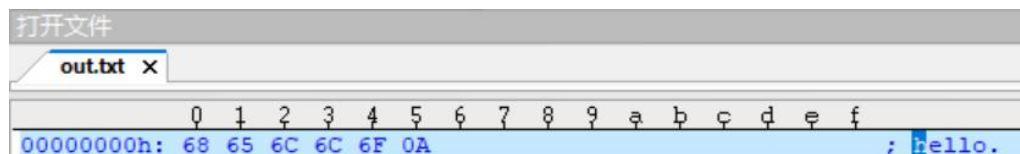
    out.close();

    return 0;
}
```

Windows下运行, out.txt是__6__字节, 用UltraEdit的16进制方式打开的贴图



Linux下运行, out.txt是__6__字节, 用UltraEdit的16进制方式打开的贴图



§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例3: 在Linux读取Windows下写的十进制文件

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
```

在Linux下运行本程序

```
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello\r" << endl; //模拟Windows格式
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
```

在Linux下运行本程序

```
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in | ios::binary);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

本例说明，在Linux下读取Windows格式的文件，要注意0D的处理

Linux下运行，输出结果是：6 6
 -1 -1

说明：

1、in.getline读到__\n__就结束了，__\n__被读掉，因此in.peek()读到了__EOF(-1)__。

2、strlen(str)是__6__，最后一个字符是__\r__

Linux下运行，输出结果是：5 5
 -1 -1

说明：

1、in.getline读到__\n__就结束了，__\n__被读掉，因此in.peek()读到了__EOF(-1)__。

2、strlen(str)是__5__，最后一个字符是__o__

§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例4: 用十进制方式写入含\0的文件, 观察文件长度

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\0\x61\x62\x63" << endl;
    out.close();

    return 0;
}
```

Windows下运行, out.txt的大小是__5__字节, Linux下运行, out.txt的大小是__4__字节

为什么? \0标志字符串的结束, 只有ABC被写入, 占3字节(Windows和Linux相同)。Windows下, 十进制方式写入文件时endl视为0D0A, 占2字节; Linux下endl视为0A, 占1字节。

§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



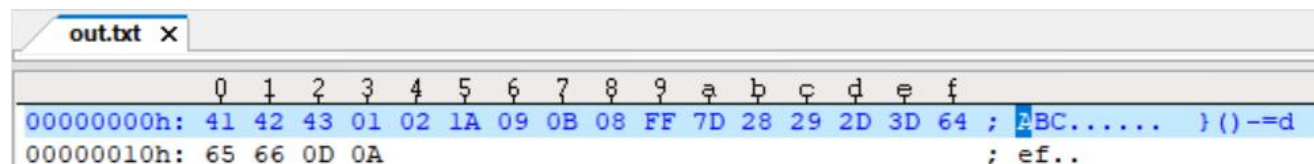
例5: 用十进制方式写入含非图形字符(ASCII码32是空格, 33-126为图形字符), 但不含\0

```
#include <iostream>
#include <fstream>
using namespace std;

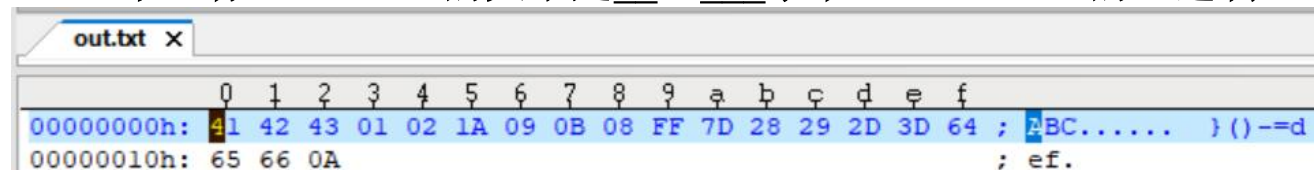
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def" << endl;
    out.close();

    return 0;
}
```

Windows下运行, out.txt的大小是__20__字节, UltraEdit的16进制显示截图为:

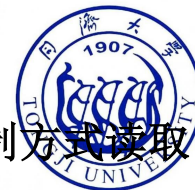


Linux下运行, out.txt的大小是__19__字节, UltraEdit的16进制显示截图为:



§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例6: 用十进制方式写入含\x1A(十进制26=CTRL+Z)和\xff(十进制255/-1, EOF的定义是-1)的文件, 并用十进制/二进制方式读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in);
    int c=0;
    while(!in.eof()) {
        in.get();
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行, 文件大小: 20
输出的c是: 6
Linux下运行, 文件大小: 19
输出的c是: 20

为什么?十进制读取, Windows下运行时, 0x1A被认为是EOF文件结束标志, 所以在in.get读到\x1A时, in.eof为真, 循环结束, c为6; Linux下0x1A不被认为是EOF

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    int c=0;
    while(!in.eof()) {
        in.get();
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行, 文件大小: 20
输出的c是: 21
Linux下运行, 文件大小: 19
输出的c是: 20

c的大小比文件大小大__1__, 原因是: 程序中第一个字符为EOF时, 还没读取到EOF, in.eof为假, 循环会多做一遍, 直到in.get读到EOF才会终止。

§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例7: 用十进制方式写入含\x1A(十进制26=CTRL+Z)的文件, 并用十进制不同方式读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\175()--def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in); //不加ios::binary
    int c=0;
    while(in.get() != EOF) {
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行, 文件大小: 19
输出的c是: 5
Linux下运行, 文件大小: 18
输出的c是: 18

为什么?十进制读取, Windows下运行时, 0x1A被认为是EOF文件结束标志, 所以在in.get读到\x1A时, 循环结束, c为5; Linux下0x1A不被认为是EOF

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\175()--def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in); //不加ios::binary
    int c=0;
    signed char ch;
    while((ch=in.get()) != EOF) {
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行, 文件大小: 19
输出的c是: 5
Linux下运行, 文件大小: 18
输出的c是: 18

为什么?和左边相同, in.get的返回值就是读取到的字符, 赋值给ch, 都是不超过1字节的正数, 结果还是一样

§ 15. 输入输出流 - Windows与Linux的

本页需填写答案



例8：用十进制方式写入含\xFF(十进制255/-1，EOF的定义是-1)的文件，并进行正确/错误读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\xff\t\v\b\175() ==def" << endl;
    out.close();

    ifstream in("out.txt", ios::in); //可加ios::binary
    int c=0;
    while(in.get() != EOF) {
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小： 19
输出的c是： 18 (加ios::binary为19)

Linux下运行，文件大小： 18
输出的c是： 18

为什么?读到0xff时，in.get的返回值为无符号数255，不会被当成EOF。Windows下，十进制读，0D0A当成一个字符\n，c比文件大小小1；二进制读，0D0A当成两字符\r\n，c和文件大小相同。Linux下，endl为0A，只有一个字符，所以文件大小小1，c和文件大小相同。

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\xff\t\v\b\175() ==def" << endl;
    out.close();

    ifstream in("out.txt", ios::in); //可加ios::binary
    int c=0;
    signed char ch;
    while((ch=in.get()) != EOF) {
        c++;
    }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小： 19
输出的c是： 5

Linux下运行，文件大小： 18
输出的c是： 5

为什么?读到0xff时，in.get的返回值为无符号数255，赋值给有符号字符类型的ch时，无符号的255转换成-1，被当成EOF，循环直接结束

综合例6~例8，结论：当文件中含字符__0x1A__时，不能用十进制方式读取，而当文件中含字符__0xFF__时，是用二/十进制方式正确读取的